

Lesson 16 — Neural Networks Basics

Now we move from traditional ML models into Neural Networks, which are the foundation of Deep Learning.

1. What is a Neural Network?

A neural network is a machine-learning model inspired loosely by how biological neurons work.

The basic idea is:

```
Input Data
  ↓
Neurons
  ↓
Layers
  ↓
Prediction
```

For example, suppose we want to predict whether a customer will leave:

```
Age  ———┐
Salary —┐
          └─┬─> Neural Network —> Churn probability
Tenure —┐
Usage  —┐
```

2. What is a Neuron?

A neuron takes inputs, multiplies them by weights, adds a bias, and then applies an activation function.

Mathematically:

$$z = w1*x1 + w2*x2 + w3*x3 + b$$

Then:

$$\text{output} = \text{activation}(z)$$

For example:

```
x1 —x w1 —┐
x2 —x w2 —┐
x3 —x w3 —┐ —> + bias —> Activation —> Output
          └─┬─┘
```

Important connection to what we just learned

Remember when you asked:

“Coefficient is decided by who?”

Here, weights are also learned by the model during training.

So:

```
Linear Regression:  
prediction = coefficient × feature + bias
```

```
Neural Network:  
prediction = weights × features + bias  
      ↓  
    activation  
      ↓  
    next layer
```

3. What is a Layer?

Neural networks contain multiple layers of neurons.

Input Layer

Receives the features.

Example:

```
Age  
Salary  
Experience
```

Hidden Layers

Perform transformations on the data.

```
Input  
  ↓  
Hidden Layer 1  
  ↓  
Hidden Layer 2  
  ↓  
Hidden Layer 3
```

Output Layer

Produces the final prediction.

```
Input → Hidden → Hidden → Output
```

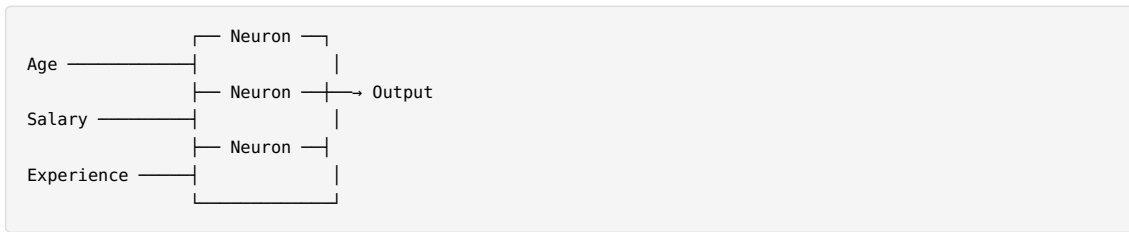
A network with many hidden layers is generally called a deep neural network.

4. Example

Suppose:

```
Input:  
x1 = Age  
x2 = Salary  
x3 = Experience
```

A simple neural network could look like:



Each connection has a weight.

The model learns those weights during training.

5. What is an Activation Function?

This is extremely important.

Without activation functions, stacking many neural-network layers would essentially behave like a more complicated linear transformation.

Activation functions introduce non-linearity.

Common activation functions:

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

So:

```
x = -5 → 0
x = 3 → 3
x = 10 → 10
```

ReLU is very commonly used in hidden layers.

Sigmoid

Output is between 0 and 1.

$$0 < \text{sigmoid}(x) < 1$$

Useful for binary classification.

For example:

$$\text{Output} = 0.87$$

can be interpreted as approximately:

87% probability of class 1

Softmax

Usually used for multi-class classification.

Example:

Cat	→ 0.10
Dog	→ 0.75
Horse	→ 0.15

The probabilities add up to 1.

6. How Does a Neural Network Learn?

This is the most important part.

Suppose the actual answer is:

Actual = 1

But the model predicts:

Prediction = 0.3

There is an error.

The neural network calculates a loss.

```
Prediction
↓
Compare with actual answer
↓
Loss
↓
Backpropagation
↓
Update weights
↓
Prediction improves
```

This process repeats many times.

7. What is Backpropagation?

Backpropagation determines how the weights contributed to the error and calculates gradients that tell the optimizer how to change them.

Think of it like:

```
Prediction
↓
How wrong were we?
↓
Which weights caused the error?
↓
Calculate gradients
↓
Adjust weights
```

Then the process repeats.

8. Gradient Descent

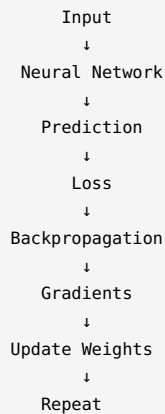
After calculating gradients, an optimizer such as gradient descent updates the weights.

Simplified:

```
w_new = w_old - learning_rate × gradient
```

The model keeps adjusting the weights to reduce the loss.

So the complete learning process is:



This connects directly to your previous question about loss.

9. What is an Epoch?

An epoch means the model has gone through the entire training dataset once.

For example:

```
Training data = 1,000 samples

1 epoch → model sees all 1,000 samples once
10 epochs → model sees them 10 times
```

More epochs don't automatically mean a better model.

Too many can cause overfitting, which connects to Lesson 15.

10. Neural Network in Scikit-Learn

For a basic neural network classifier:

```
from sklearn.neural_network import MLPClassifier
```

MLP means:

Multi-Layer Perceptron

Example:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

```
# Load data
```

```

data = load_iris()

X = data.data
y = data.target

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create neural network
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation="relu",
    max_iter=1000,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

11. Understand MLPClassifier

This part is important:

```

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation="relu",
    max_iter=1000,
    random_state=42
)

```

hidden_layer_sizes=(10,)

Means:

```

Input
↓
10 neurons
↓
Output

```

The comma is important because (10,) is a Python tuple.

Two hidden layers

```
hidden_layer_sizes=(10, 5)
```

means:

```
Input
↓
10 neurons
↓
5 neurons
↓
Output
```

activation="relu"

Uses ReLU in the hidden layers.

max_iter=1000

Maximum number of training iterations allowed.

random_state=42

Makes the result reproducible.

12. Why Do We Scale Neural Network Data?

Neural networks are generally easier to train when features are on comparable scales.

For example:

```
Age      → 20–60
Salary   → 20,000–500,000
Experience → 0–30
```

Salary has a much larger numerical scale.

So we use:

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Important: fit the scaler only on training data, then transform the test data.

13. Neural Network vs Traditional ML

You have now learned many models:

Model	Main idea
Linear Regression	Fit a linear relationship
Logistic Regression	Classification using probabilities
KNN	Look at nearby points
Decision Tree	Split data using rules
Random Forest	Many decision trees
Gradient Boosting	Sequentially improve trees
K-Means	Find groups
PCA	Reduce dimensions

The Big Picture

You should now understand the basic neural-network pipeline:

```
TRAINING
↓
Input Features
↓
Weighted Sum
↓
Activation
↓
Hidden Layers
↓
Prediction
↓
Calculate Loss
↓
Backpropagation
↓
Calculate Gradients
↓
Update Weights
↓
Repeat
```

One sentence to remember:

A neural network learns by adjusting its weights to minimize the loss through backpropagation and optimization.

And this connects directly to what you learned in Lesson 15:

Loss tells the model how wrong it is → gradients tell it how to change → weights are updated → predictions improve.

Next in Lesson 16, the most important thing to understand is forward propagation, backpropagation, neurons, weights, bias, and activation functions with an actual numerical example.

Lesson 16 — Neural Networks Basics: Part 2

Now let's go inside a neural network and understand exactly how a neuron produces an output and how the model learns its weights.

1. One Neuron

Suppose we have three inputs:

```
x1 = 2
x2 = 3
x3 = 1
```

And the model has learned:

```
w1 = 0.5
w2 = 0.2
w3 = -0.4

b = 0.1
```

The neuron first calculates:

```
z = w1*x1 + w2*x2 + w3*x3 + b
```

Substitute the values:

```
z = (0.5)(2) + (0.2)(3) + (-0.4)(1) + 0.1
z = 1 + 0.6 - 0.4 + 0.1
z = 1.3
```

So before activation:

```
z = 1.3
```

2. Apply Activation Function

Suppose we use ReLU.

```
ReLU(z) = max(0, z)
```

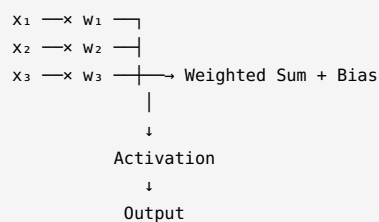
Therefore:

```
ReLU(1.3) = 1.3
```

So the neuron's final output is:

```
1.3
```

The complete neuron is:



3. Why Do We Need Weights?

Weights determine how important each input is.

Suppose:

```
w1 = 5
w2 = 0.1
w3 = 0.01
```

Then the neuron is giving much more importance to x_1 .

So:

Weight = how strongly a feature influences a neuron.

This is similar to coefficients in linear regression.

4. What is Bias?

Bias gives the neuron additional flexibility.

The formula is:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$$

Without bias:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2$$

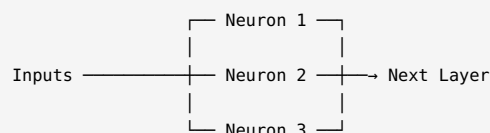
With bias:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

Think of bias as allowing the neuron to shift its decision/activation.

5. Multiple Neurons

Now suppose we have 3 neurons:



Each neuron has its own weights and bias.

For example:

Neuron 1:
 w_1, w_2, w_3, b_1

Neuron 2:
 w_1, w_2, w_3, b_2

Neuron 3:
 w_1, w_2, w_3, b_3

So different neurons can learn different patterns.

6. What Does a Hidden Layer Do?

This is where neural networks become powerful.

Imagine you're detecting whether an image contains a face.

The early layers might learn simple patterns:

Layer 1
→ edges

Then:

Layer 2
→ shapes

Then:

Layer 3
→ eyes / nose / mouth patterns

Then:

Output
→ face / not face

The network learns these representations automatically from data.

7. Forward Propagation

When data moves:

Input
↓
Layer 1
↓
Layer 2
↓
Output

this is called forward propagation (or forward pass).

For example:

Input
[2, 3, 1]
↓
Hidden Layer
[1.3, 0.7, 2.1]
↓
Output Layer
[0.82]

The final value might represent:

82% probability → class 1

8. Then We Calculate Loss

Suppose:

Actual = 1
Prediction = 0.82

The prediction isn't perfect.

So we calculate a loss.

For binary classification, a common loss is binary cross-entropy:

$$\text{Loss} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

You don't need to memorize the formula yet.

Just understand:

```
Good prediction
  ↓
Small loss

Bad prediction
  ↓
Large loss
```

9. Backpropagation

Now comes the important part.

The network asks:

“Which weights contributed to this error, and by how much?”

It calculates gradients.

For example:

Weight	Gradient
w_1	+0.8
w_2	-0.2
w_3	+0.4

These gradients tell the optimizer the direction in which each weight should change to reduce the loss.

10. Weight Update

Suppose:

```
old weight = 0.5
gradient   = 0.8
learning rate = 0.1
```

The simplified update is:

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} \times \text{gradient}$$

Therefore:

$$\begin{aligned} w_{\text{new}} &= 0.5 - (0.1)(0.8) \\ w_{\text{new}} &= 0.42 \end{aligned}$$

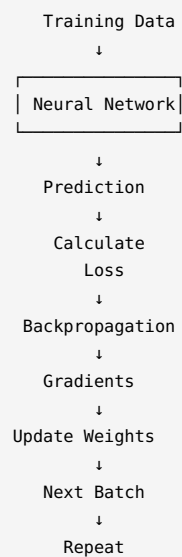
So:

```
Before:  
w = 0.50  
  
After:  
w = 0.42
```

The network keeps doing this for all its weights.

11. The Complete Learning Process

This is the part I want you to remember:



Eventually:

```
Loss ↓  
Prediction quality ↑
```

12. What is Learning Rate?

The learning rate controls how big each weight update is.

Small learning rate

```
0.001
```

Small steps:

```
Weight  
↓  
small update  
↓  
small update  
↓  
small update
```

Training can be slow.

Large learning rate

0.5

Large steps:

```
Weight
↓
BIG update
↓
BIG update
```

The model may overshoot good values and training can become unstable.

So we want an appropriate learning rate.

13. What is an Optimizer?

The optimizer is responsible for updating the model's weights using the gradients.

Common optimizers include:

- Gradient Descent
- SGD
- Adam
- RMSprop

You'll commonly encounter Adam when working with neural networks.

Conceptually:

```
Loss
↓
Gradient
↓
Optimizer
↓
New weights
```

14. One Very Important Connection

You previously learned:

Linear Regression

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

Neural networks start with a similar calculation:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

But then add:

`activation(z)`

And stack many such layers:

```
Input
↓
Weighted Sum
↓
Activation
↓
Weighted Sum
↓
Activation
↓
Weighted Sum
↓
Output
```

That's the basic idea behind a neural network.

What You Should Know After This Part

Make sure these terms are clear:

Term	Meaning
Neuron	Performs weighted calculation + activation
Weight	Learned importance of an input
Bias	Learned shift added to weighted sum
Activation	Adds non-linearity
Layer	Collection of neurons
Forward propagation	Input → prediction
Loss	Measures prediction error
Backpropagation	Calculates gradients for weight updates
Gradient	Direction/amount for changing weights
Optimizer	Uses gradients to update weights
Epoch	One complete pass through training data
Learning rate	Controls size of weight updates

The core idea:

Neural networks learn by repeatedly making predictions, calculating loss, using backpropagation to calculate gradients, and updating their weights.

Next we'll do Lesson 16 practical coding with `MLPClassifier`, visualize the neural network's training loss, and understand `hidden_layer_sizes`, `activation`, `solver`, `learning_rate`, and `max_iter` practically.

Lesson 16 — Neural Networks: Practical Coding

Now let's actually build and train a neural network using `scikit-learn`.

We'll use the Iris dataset, because it's small and easy to understand.

1. Import Everything

```
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
```

2. Load the Dataset

```
data = load_iris()

X = data.data
y = data.target

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Output:

```
X shape: (150, 4)
y shape: (150,)
```

Meaning:

```
150 samples
4 features
```

The four features are:

```
sepal length
sepal width
petal length
petal width
```

And there are 3 classes:

```
0 → setosa
1 → versicolor
2 → virginica
```

3. Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

We're using:

```
80% → training
20% → testing
```

So approximately:

```
120 → training
30 → testing
```

4. Scale the Features

Neural networks generally work better when features are on comparable scales.

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Important

We use:

```
scaler.fit_transform(X_train)
```

but only:

```
scaler.transform(X_test)
```

on test data.

Why?

Because the test set must remain unseen during training.

5. Create the Neural Network

```
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation="relu",
    solver="adam",
    max_iter=1000,
    random_state=42
)
```

Now let's understand every parameter.

hidden_layer_sizes=(10,)

This means:

```
Input layer
  ↓
10 neurons
  ↓
Output layer
```

We have 4 input features:

```
4 inputs
  ↓
10 hidden neurons
  ↓
3 outputs
```

Because Iris has 3 classes.

What if we use:

```
hidden_layer_sizes=(10, 5)
```

Then:

```
Input
↓
10 neurons
↓
5 neurons
↓
Output
```

So:

```
(10,)      → 1 hidden layer
(10, 5)    → 2 hidden layers
(10, 5, 3) → 3 hidden layers
```

6. `activation="relu"`

We learned this earlier:

```
ReLU(x) = max(0, x)
```

Example:

```
-5 → 0
-2 → 0
0 → 0
3 → 3
7 → 7
```

ReLU introduces non-linearity into the network.

7. `solver="adam"`

This determines how the model updates its weights.

Remember:

```
Loss
↓
Gradients
↓
Optimizer
↓
Update weights
```

`adam` is an optimization algorithm commonly used for neural networks.

You don't need to understand the mathematics of Adam yet.

Just remember:

Adam is an optimizer that helps update the neural network's weights efficiently.

8. `max_iter=1000`

This tells the model the maximum number of training iterations.

```
max_iter=1000
```

It does not necessarily mean exactly 1000 epochs.

It's better to think of it as the maximum number of optimization iterations allowed by the solver.

9. Train the Network

```
model.fit(X_train, y_train)
```

This is where the learning happens.

Internally, the model approximately does:

```
Training data
↓
Forward pass
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Gradients
↓
Adam optimizer
↓
Update weights
↓
Repeat
```

10. Make Predictions

```
y_pred = model.predict(X_test)

print(y_pred)
```

You might see:

```
[1 0 2 1 1 0 2 ...]
```

These are predicted class labels.

11. Check Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

You might get something around:

```
Accuracy: 0.96
```

That means approximately:

```
96% of test samples were correctly classified
```

12. Classification Report

```
print(classification_report(y_test, y_pred))
```

You'll get metrics such as:

```
precision
recall
f1-score
support
```

You already learned these in Lesson 5, so this is another application of those evaluation metrics.

13. See the Training Loss

This is very useful for understanding what the neural network is actually doing.

`MLPClassifier` stores the loss after each iteration:

```
print(model.loss_curve_)
```

You can visualize it:

```
import matplotlib.pyplot as plt

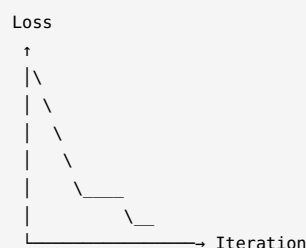
plt.figure(figsize=(8, 5))

plt.plot(model.loss_curve_)

plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Neural Network Training Loss")

plt.show()
```

You should generally see something like:



The exact curve will vary.

The important idea is:

Loss decreases ↓

as training progresses.

14. Complete Code

Here's the complete runnable version:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# =====
# 1. Load dataset
# =====

data = load_iris()

X = data.data
y = data.target

print("X shape:", X.shape)
print("y shape:", y.shape)

# =====
# 2. Train-test split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# =====
# 3. Feature scaling
# =====

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 4. Create Neural Network
# =====

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation="relu",
    solver="adam",
    max_iter=1000,
    random_state=42
)
```

```

# =====
# 5. Train
# =====

model.fit(X_train, y_train)

# =====
# 6. Predict
# =====

y_pred = model.predict(X_test)

# =====
# 7. Evaluate
# =====

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# 8. Training loss
# =====

print("\nFinal Loss:", model.loss_)

print("Number of iterations:", model.n_iter_)

# =====
# 9. Plot loss curve
# =====

plt.figure(figsize=(8, 5))

plt.plot(model.loss_curve_)

plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Neural Network Training Loss")

plt.show()

```

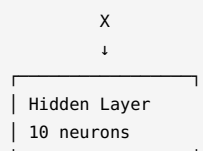
15. What Happened Internally?

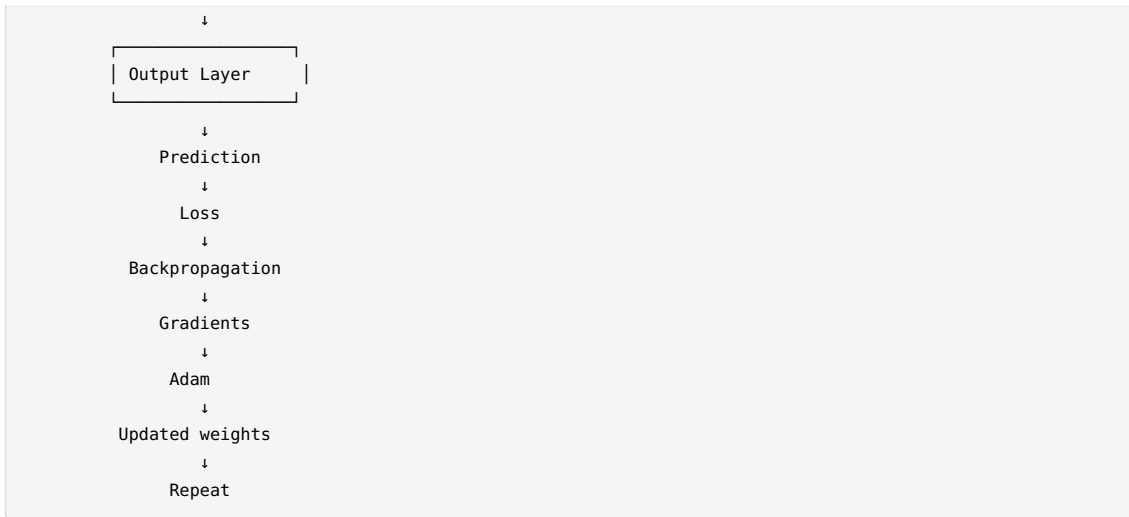
When you run:

```
model.fit(X_train, y_train)
```

don't imagine that scikit-learn simply "calculates" the answer.

It performs an iterative learning process:





This is the heart of neural-network training.

🔥 16. Connection to Lesson 15

This is particularly important because you're coming from regularization and overfitting.

A neural network can also overfit.

For example:

```
Training accuracy = 99%
Testing accuracy  = 75%
```

That's a warning sign.

Neural networks can be controlled using things such as:

- network size
- regularization
- early stopping
- learning rate
- amount of training
- more training data

So overfitting isn't only a problem for linear regression or decision trees.

🎯 What You've Learned So Far

You now understand the basic neural-network workflow:

```
Data
↓
Scaling
↓
Neural Network
↓
Forward Propagation
↓
Prediction
↓
Loss
↓
Backpropagation
```

```
graph TD; A[↓] --> B[Gradients]; B --> C[↓]; C --> D[Optimizer]; D --> E[↓]; E --> F[Weight Updates]; F --> G[↓]; G --> H[Repeat]; H --> I[↓]; I --> J[Final Model];
```

↓
Gradients
↓
Optimizer
↓
Weight Updates
↓
Repeat
↓
Final Model

And the most important distinction:

Weights are learned by the neural network; `hidden_layer_sizes`, activation, learning settings, etc. are hyperparameters chosen by us.